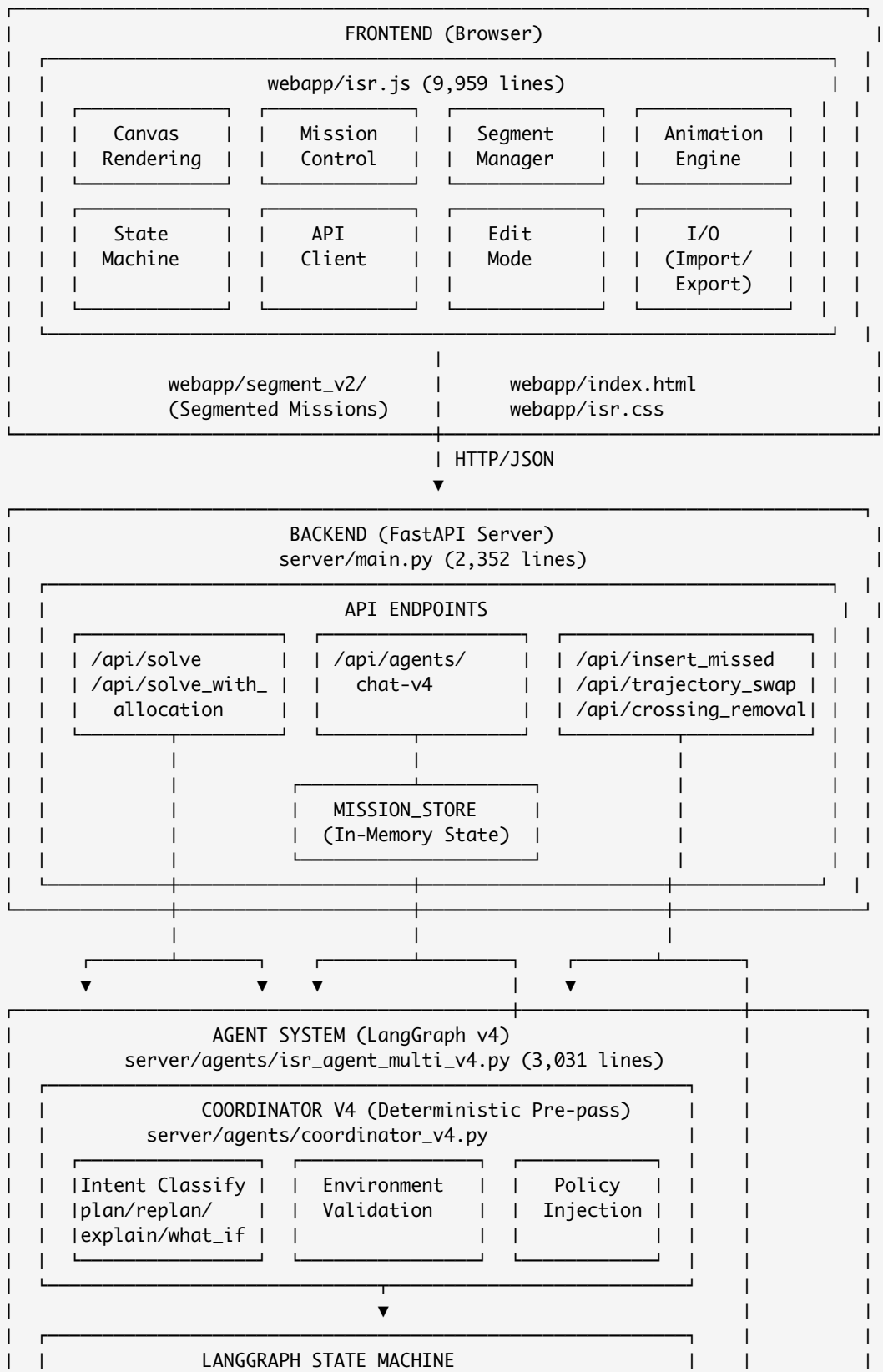
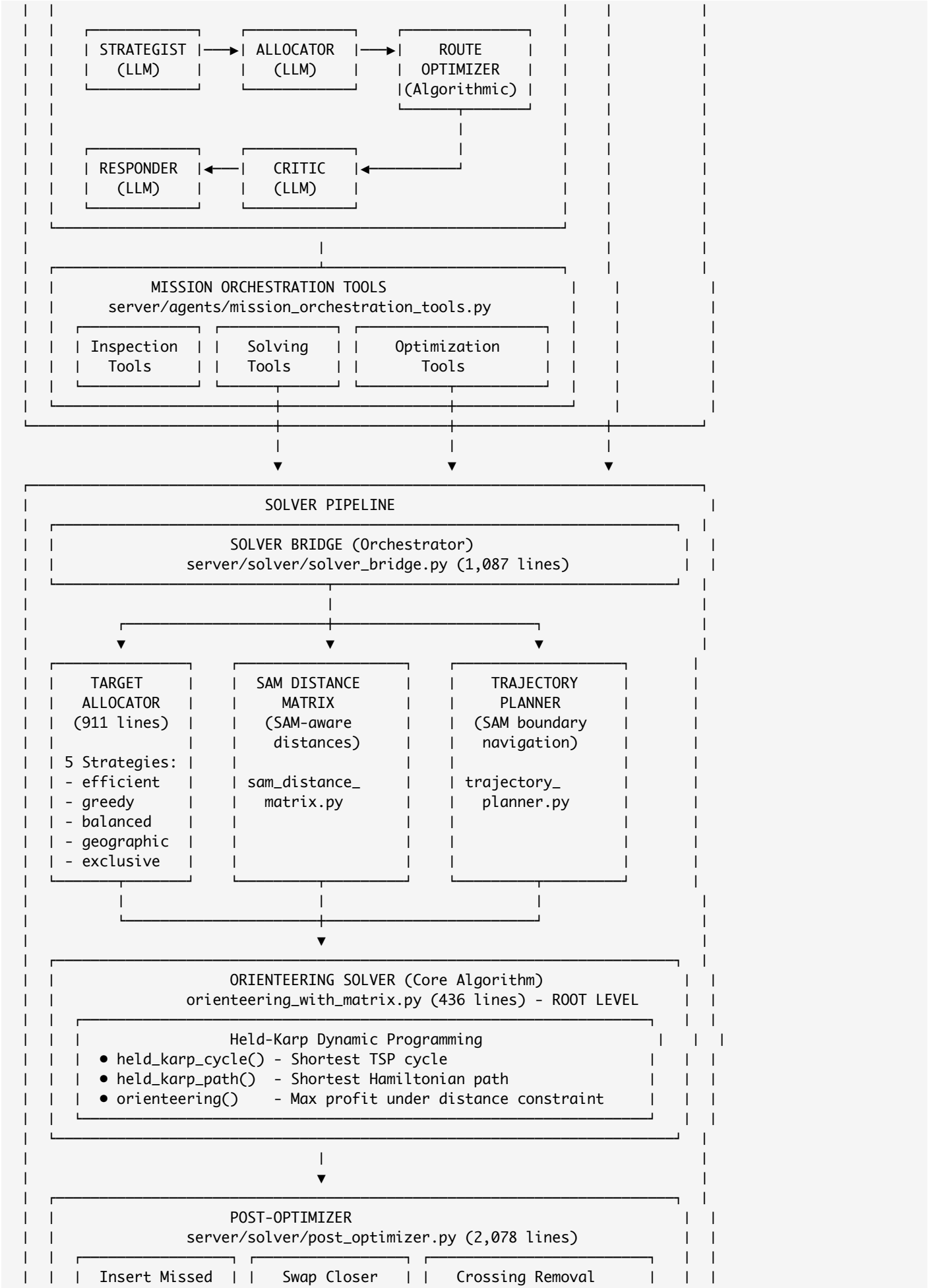
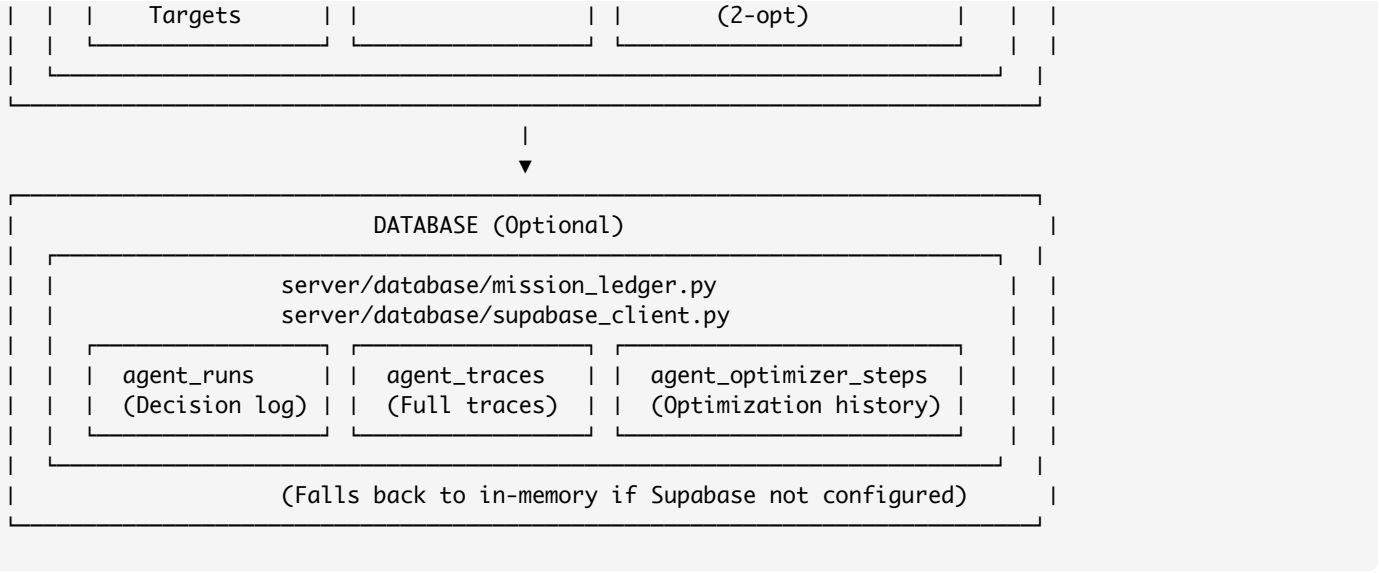
ISR Mission Planner - Architecture Diagram

ISR Mission Planner - Architecture Diagram

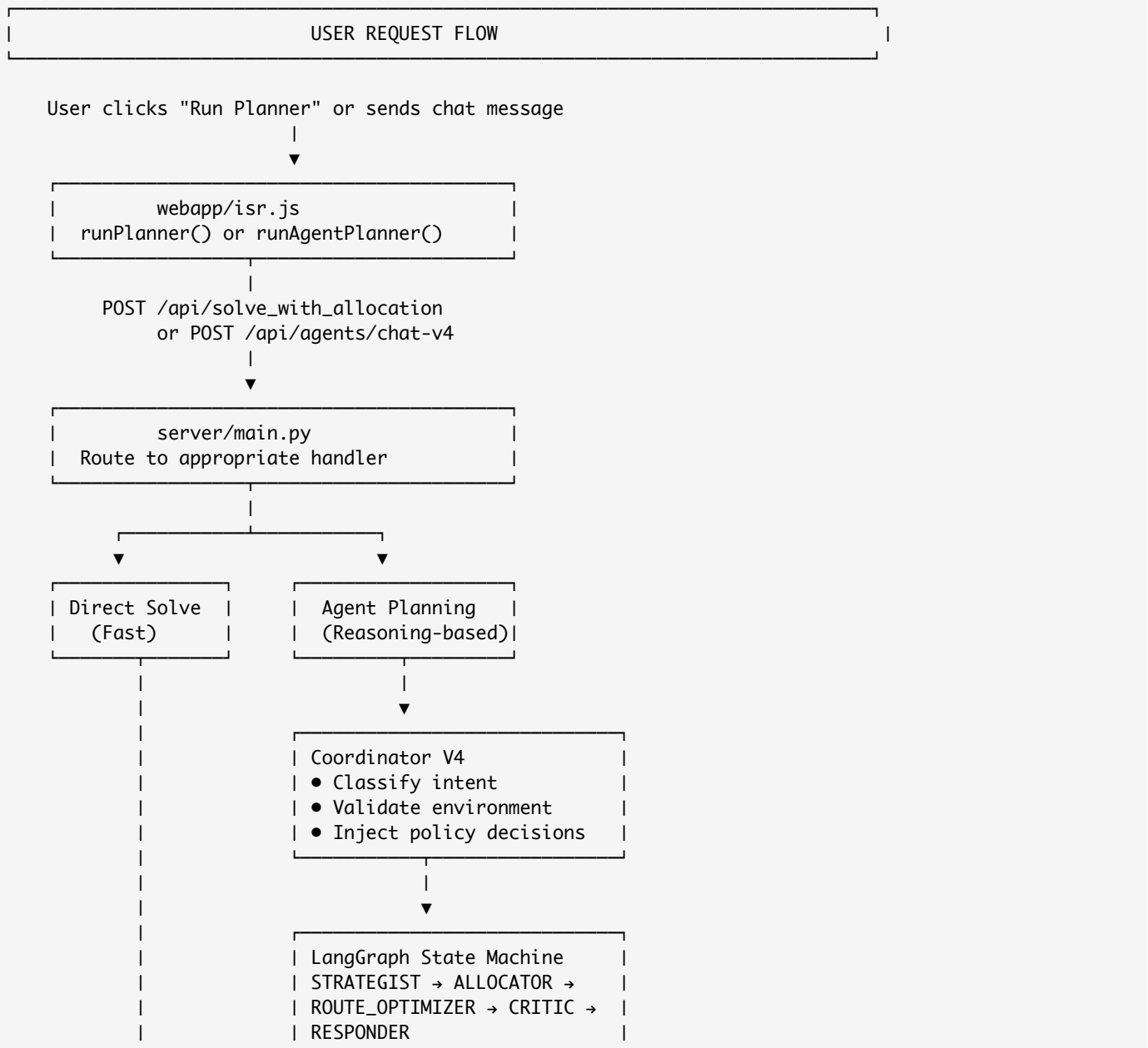
High-Level System Architecture

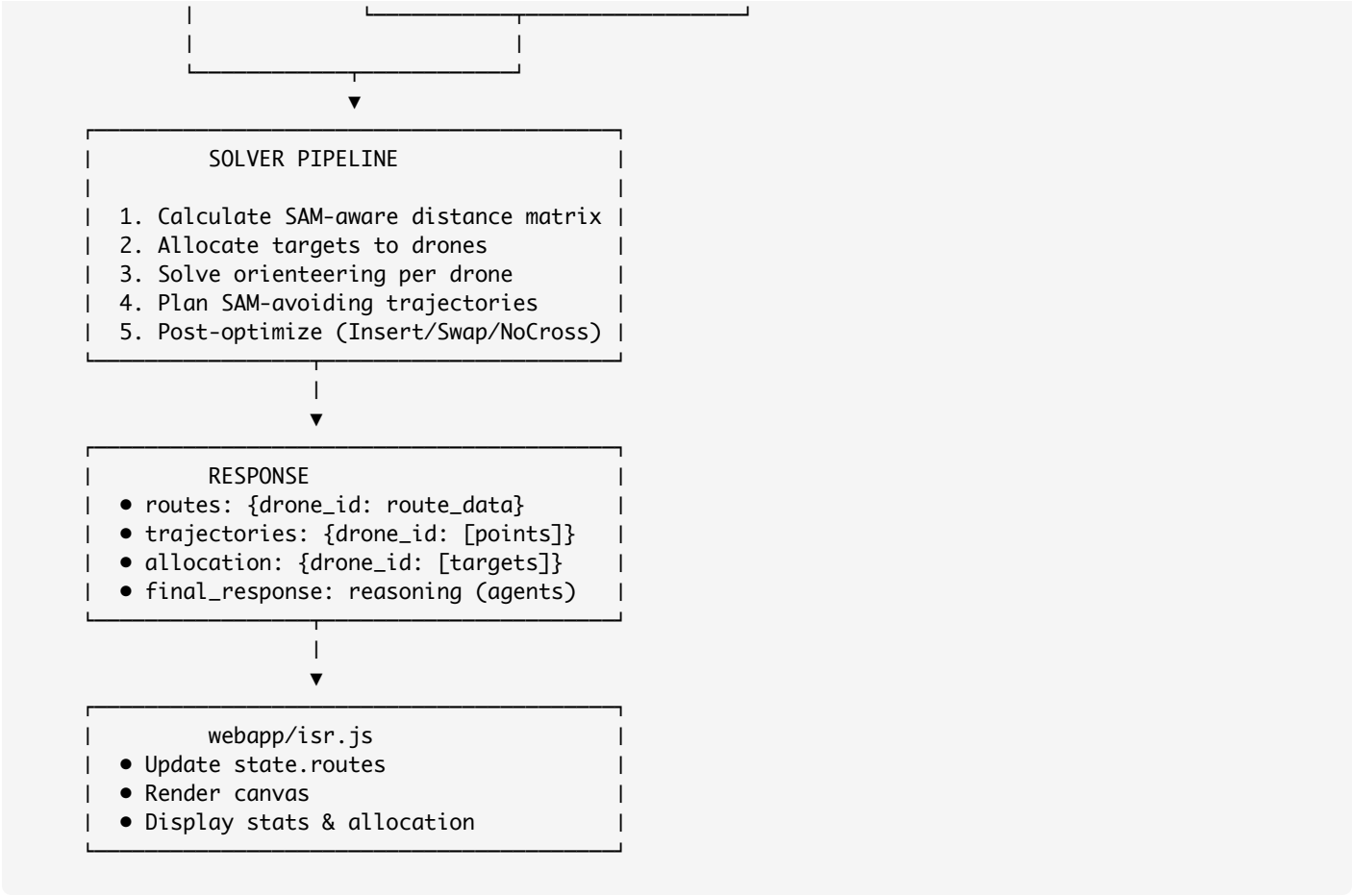



STRATEGIST (LLM)
ALLOCATOR (LLM)
ROUTE OPTIMIZER (Algorithmic)
RESPONDER (LLM)
CRITIC (LLM)
MISSION ORCHESTRATION TOOLS
server/agents/mission_orchestration_tools.py
Inspection Tools
Solving Tools
Optimization Tools
SOLVER PIPELINE
SOLVER BRIDGE (Orchestrator)
server/solver/solver_bridge.py (1,087 lines)
TARGET ALLOCATOR (911 lines)
5 Strategies:
- efficient
- greedy
- balanced
- geographic
- exclusive
SAM DISTANCE MATRIX (SAM-aware distances)
sam_distance_matrix.py
TRAJECTORY PLANNER (SAM boundary navigation)
trajectory_planner.py
ORIENTEERING SOLVER (Core Algorithm)
orienteering_with_matrix.py (436 lines) - ROOT LEVEL
Held-Karp Dynamic Programming
• held_karp_cycle() - Shortest TSP cycle
• held_karp_path() - Shortest Hamiltonian path
• orienteering() - Max profit under distance constraint
POST-OPTIMIZER
server/solver/post_optimizer.py (2,078 lines)
Insert Missed
Swap Closer
Crossing Removal

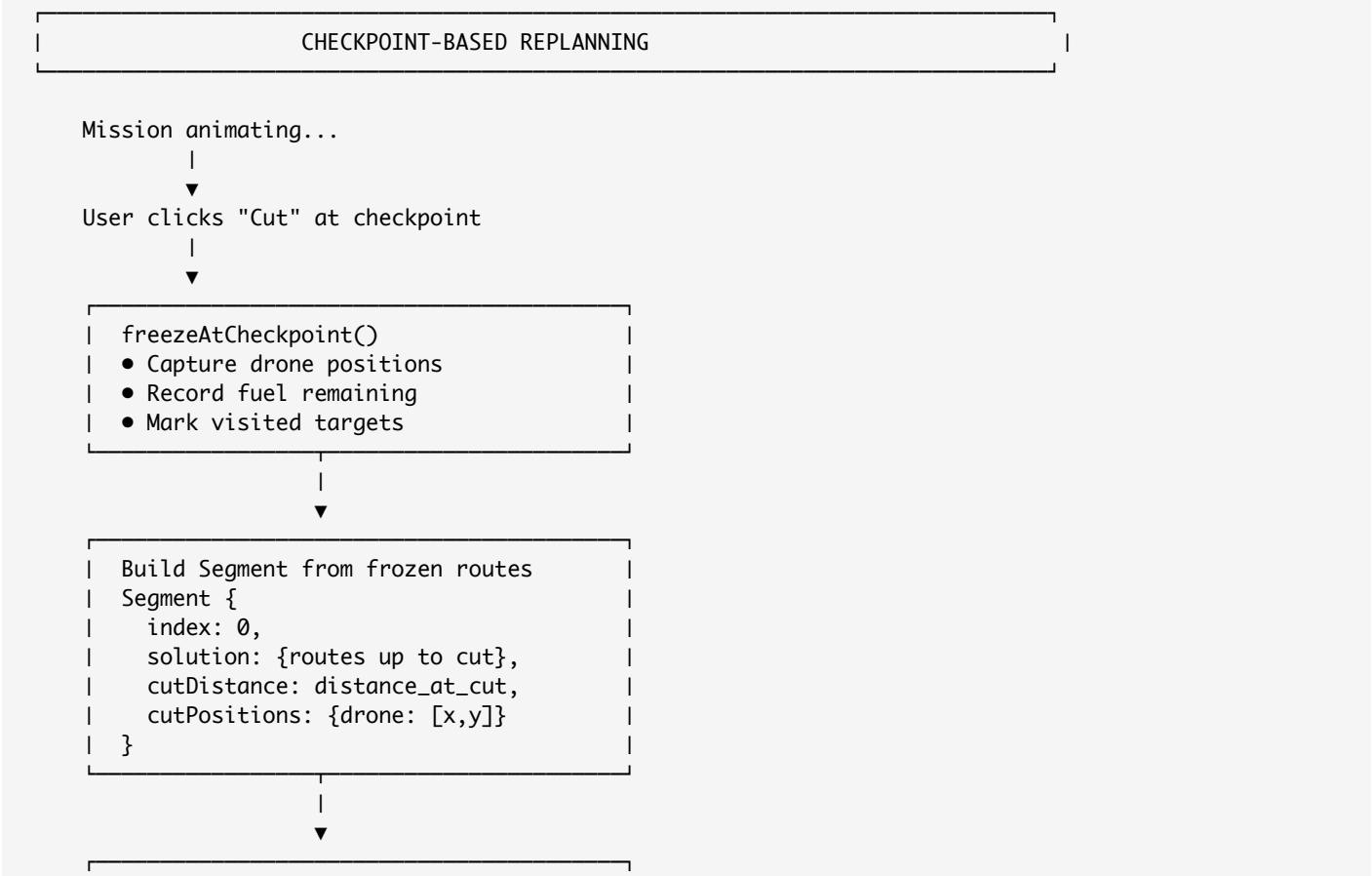


Data Flow: Mission Planning Request





Segmented Mission System



```
    |  Replan remaining mission               |
    |  POST /api/solve_with_allocation        |
    |  • Filter out visited targets           |
    |  • Set synthetic_starts to cut positions|
    |  • Return new routes for remainder      |
    └─────────────────┬───────────────────────┘
                      |
                      ▼
    ┌─────────────────────────────────────────┐
    |  Build Segment from new solution        |
    |  Segment {                              |
    |    index: 1,                            |
    |    solution: {new routes},              |
    |    ...                                  |
    |  }                                      |
    └─────────────────┬───────────────────────┘
                      |
                      ▼
    ┌─────────────────────────────────────────┐
    |  committedSegments = [Segment0, Segment1]|
    |  Continue animation from Segment 1      |
    └─────────────────────────────────────────┘
```

File Organization Summary

```
isr-planner/
├── orienteering_with_matrix.py          # Core Held-Karp solver (ROOT LEVEL!)
├── run_planner.sh                        # Startup script
├── .env                                  # Environment variables
|
├── server/
|   ├── main.py                           # FastAPI app + ALL endpoints (2,352 lines)
|   |
|   ├── agents/
|   |   ├── isr_agent_multi_v4.py        # ACTIVE v4 agent (3,031 lines)
|   |   ├── coordinator_v4.py            # Deterministic pre-pass
|   |   ├── mission_orchestration_tools.py # Agent-accessible tools
|   |   ├── mission_tools.py             # Mission analysis helpers
|   |   └── versions/                    # Legacy agents (v1, v2, v3)
|   |
|   ├── solver/
|   |   ├── solver_bridge.py             # Main orchestrator (1,087 lines)
|   |   ├── target_allocator.py          # 5 allocation strategies (911 lines)
|   |   ├── post_optimizer.py            # Insert/Swap/NoCross (2,078 lines)
|   |   ├── sam_distance_matrix.py       # SAM-aware distances
|   |   └── trajectory_planner.py        # SAM boundary navigation
|   |
|   ├── database/
|   |   ├── mission_ledger.py            # Decision trace tables
|   |   └── supabase_client.py           # Database connection
|   |
|   └── memory/
|       ├── constraints/                 # Constraint parsing
|       └── sessions/                    # Session state
|
├── webapp/
|   ├── index.html                       # Main HTML
|   ├── isr.js                           # MONOLITHIC frontend (9,959 lines)
|   ├── isr.css                          # Styling
|   └── segment_v2/                      # Segmented mission system
|       ├── Segment.js
|       ├── SegmentBuilder.js
|       └── MissionReplay.js
```

```
|
└── docs/
    ├── SYSTEM_ARCHITECTURE.md           # Segment system design
    ├── AGENTIC_SYSTEM_DESCRIPTION.md    # v4 agent architecture
    └── MODULAR_ARCHITECTURE.md          # Planned refactoring
```

Key Technologies

Layer	Technology
Frontend	Vanilla JavaScript, HTML5 Canvas
Backend	Python FastAPI
Agent Framework	LangGraph v4
AI Model	Claude (Anthropic)
Solver Algorithm	Held-Karp Dynamic Programming
Database (Optional)	Supabase (PostgreSQL)

PDF Generation Plan

To generate a PDF of this architecture diagram:

1. **Install pandoc** (if not already installed):

```
brew install pandoc
brew install --cask basictex  # For PDF output via LaTeX
```

2. **Generate PDF**:

```
pandoc /Users/kamalali/.claude/plans/snoopy-juggling-kurzweil.md \
  -o /Users/kamalali/isr-planner/isr-planner/docs/ISR_ARCHITECTURE_DIAGRAM.pdf \
  --pdf-engine=pdflatex \
  -V geometry:margin=1in \
  -V monofont="Courier New"
```

   Alternative (without LaTeX):

```
pandoc /Users/kamalali/.claude/plans/snoopy-juggling-kurzweil.md \
  -o /Users/kamalali/isr-planner/isr-planner/docs/ISR_ARCHITECTURE_DIAGRAM.html
# Then print to PDF from browser
```

Output location: `docs/ISR_ARCHITECTURE_DIAGRAM.pdf`